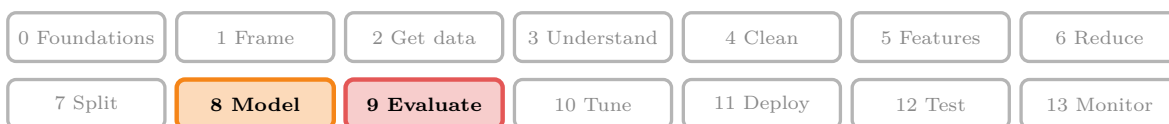


Why Random Forests Work: Bias and Variance

Contents

1. Overview	2
2. Two kinds of algorithm	3
3. How a random forest lowers the variance	3
4. Seeing it in classification	4
5. Seeing it in regression	5
6. Summary	6
7. Key terms	6

Where this fits: Pipeline map steps 8 (Model), 9 (Evaluate). See the [Course map](#).



- **Builds on:** Polynomial regression ([Note 61](#)); Overfitting ([Note 91](#)); Underfitting ([Note 91](#)); Hyperparameter tuning ([Note 98](#)); Decision trees ([Note 100](#)); Feature importance ([Note 100](#)).
- **Leads to:** Boosting ([Note 119](#)); Balanced random forest ([Note 133](#)).
- **Compare with:** K-nearest neighbours ([Note 91](#)); Bagging ([Note 107](#)).

1. Overview

Key point: A random forest takes fully grown trees, which have low bias and high variance, and averages away most of the variance. The result has low bias and low variance, the combination single models rarely reach.

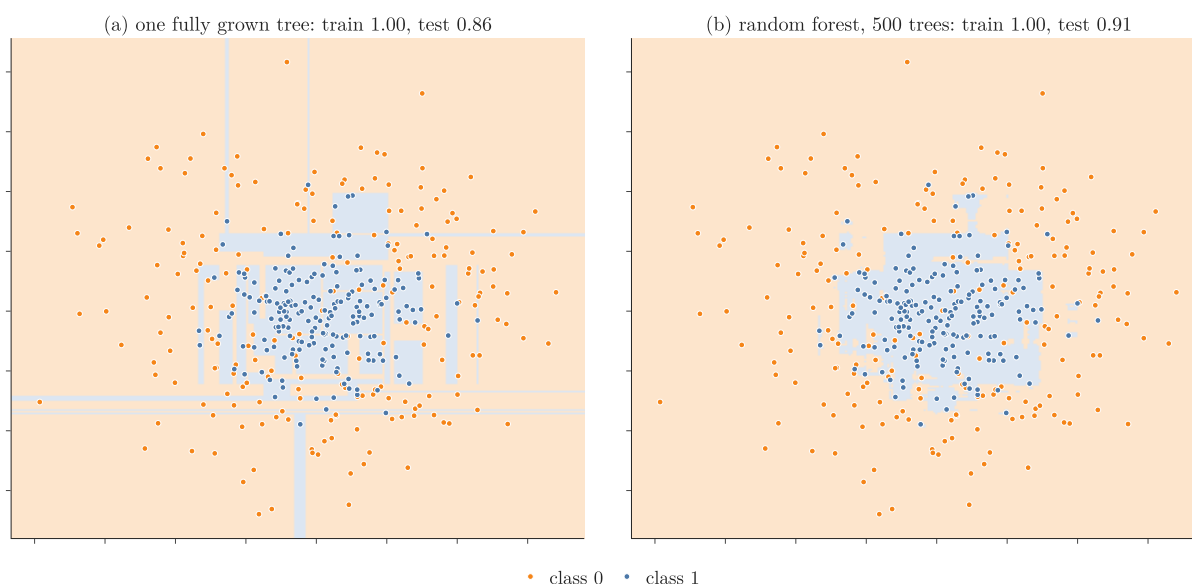


Figure 1: Concentric circles: (a) one fully grown tree draws boxes around single points; (b) a random forest of 500 trees draws one smooth inner region. Titles give the accuracy on the training and test sets

Figure 1 shows the whole story. One fully grown tree (a) carves out thin strips and small boxes for single points: it overfits. A forest of 500 such trees (b) keeps the shape of the data, a blue disc inside an orange ring, and drops most of the strips. Its test accuracy rises from 0.86 to 0.91.

This Note explains why, using the bias-variance trade-off, and shows the same effect on a regres-

sion problem. The Notebook (`notebook.ipynb`) computes every number; `images/figs.py` draws the figures.

2. Two kinds of algorithm

Key point: Most algorithms are either low bias, high variance (they overfit) or high bias, low variance (they underfit); we want low bias and low variance.

From the [bias-variance Note](#): **bias** is a model's inability to fit even its training data, and **variance** is how much its predictions change when the training data changes. We want both low, but in a single model lowering one usually raises the other.

So most algorithms fall into one of two groups:

Group	Behaviour	Examples
Low bias, high variance	very good on the training data, much worse on new data (overfitting)	a fully grown decision tree (no <code>max_depth</code>); an SVM with a very flexible boundary; KNN with a small <code>k</code>
High bias, low variance	mediocre on the training data, about the same on new data (underfitting)	linear regression; logistic regression, in some cases

Neither group gives what we want: low bias **and** low variance. A random forest gets there by starting from the first group.

3. How a random forest lowers the variance

Key point: Each tree is fully grown, so bias stays low; each tree sees a different random sample, so the effect of any noisy rows is spread out and averaged away.

A random forest is built from **fully grown** trees: every base model fits its training data almost perfectly, so every one has low bias and high variance. Averaging them leaves the bias low.

The variance falls for the reason given in the [bagging Note](#), section 3.2. Suppose 100 of 10,000 rows are replaced by noisy outliers. A single tree trained on all the data must absorb all 100 and changes its logic; in a forest, each tree draws its own random sample, so one tree gets 10 of the noisy rows, another 5, another none.

No tree bears the full impact, and the vote or mean dilutes what each tree did absorb. The forest's predictions barely change when the data changes: **low variance**. That is how the forest escapes the trade-off: it lowers the variance while keeping the bias low.

4. Seeing it in classification

Key point: On noisy concentric circles, one tree scores 0.86 on the test set and a forest of 500 trees 0.91.

The data is made with `make_circles` (the [kernel trick code Note](#)): 500 points, 2 columns, two classes. The blue class (1) sits in a small disc inside the orange class (0), with plenty of noise, so the classes overlap. We train on 400 points and test on 100.

One fully grown tree (Figure 1a). Its surface has long thin strips and isolated boxes. Each exists because of a single point: a lone blue point among orange ones gets its own blue strip, although the region around it clearly belongs to orange.

- Training accuracy: **1.00**, every training point is classified correctly: low bias.
- Test accuracy: **0.86**. A new sample from the same pattern would produce different strips in different places: high variance. This is overfitting.

A random forest of 500 trees (Figure 1b). Most strips and boxes are gone. The surface captures the true nature of the data: a blue region inside an orange one.

- Training accuracy: still **1.00**.
- Test accuracy: **0.91**: the variance has dropped.

Python: The two models.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
```

```
tree = DecisionTreeClassifier(random_state=42)
tree.fit(X_train, y_train)
rf = RandomForestClassifier(n_estimators=500,
                           random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
rf.score(X_test, y_test)      # 0.91
```

`n_estimators` is the number of trees and `n_jobs=-1` trains them on every CPU core (both from the [bagging classifier Note](#)). The surfaces are drawn on a grid of points, as in the [KNN Note](#), section 5.

Extra: One might expect the forest to give up a little training accuracy in exchange for the lower variance. Here it does not: each fully grown tree sees about 63% of the training rows, and for every training point the many trees that did see it outvote the rest. On noisier data the forest's training accuracy can fall slightly below 1, a small rise in bias.

5. Seeing it in regression

Key point: On a noisy curve, one tree chases every point (test MSE 0.0192); a forest of 1,000 trees stays closer to the true pattern (test MSE 0.0140).

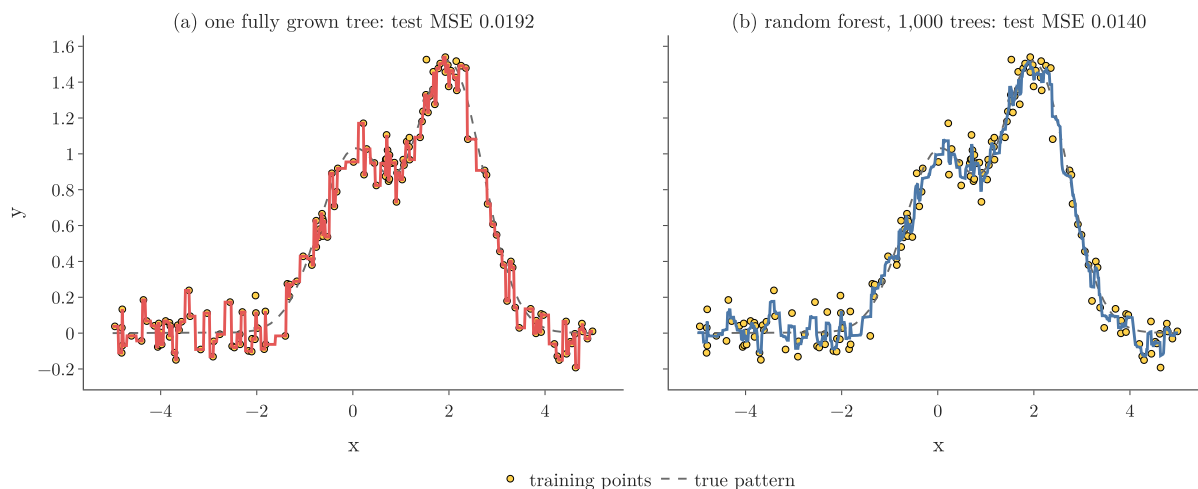


Figure 2: Two bumps plus noise: (a) one fully grown regression tree; (b) a random forest of 1,000 trees. The dashed curve is the true pattern

The data is the two-bumps curve of the [bagging regressor Note](#): one input column, an output made of two Gaussian bumps plus noise. The dashed curve in Figure 2 is the true pattern; we train on 150 points and test on 1,000.

One fully grown tree (Figure 2a, red) tries to pass through every training point, outliers included. Its training error is 0: low bias. But it jumps up and down to reach every extreme point, so it is wrong on new points: high variance. Test **mean squared error (MSE)** (the [regression metrics Note](#)): **0.0192**.

A random forest of 1,000 trees (Figure 2b, blue) is smoother. It no longer reaches every outlier and stays closer to the dashed curve. Its training MSE rises a little, from 0 to 0.0018, while its test MSE falls to **0.0140**, about 27% lower.

Extra: With a single input column, a random forest regressor is the same as bagging trees on full-size bootstrap samples: there are no columns to sample, and `RandomForestRegressor` uses every column at every split by default anyway. So Figure 2 shows the same effect as the [bagging regressor Note](#).

Older code often prints `np.sum((y_test - pred) ** 2)` and calls it MSE. That is the **sum** of squared errors (19.2 and 14.0 here, over 1,000 test points); the mean divides by the number of points. The comparison between models comes out the same, but only the mean can be compared across test sets of different sizes.

6. Summary

	One fully grown tree	Random forest
Bias	low	low
Variance	high	low
Circles: train / test accuracy	1.00 / 0.86	1.00 / 0.91
Curve: train / test MSE	0.0000 / 0.0192	0.0018 / 0.0140

- We want low bias and low variance, but single models usually trade one for the other.
- Fully grown trees are low bias, high variance: they overfit.
- A random forest averages many such trees, each trained on a different random sample, so noisy rows are spread out and their effect averages away.
- The result keeps the low bias and cuts the variance: smoother boundaries and curves, better scores on new data.

7. Key terms

Term	Meaning
Low bias, high variance algorithm	An algorithm that fits its training data very well but changes a lot with the data, such as a fully grown tree; it overfits
High bias, low variance algorithm	An algorithm too simple to fit the training data well but stable across samples, such as linear regression; it underfits